# Homework 11

Solutions will be available after Monday, November 9, 2026.

## Question 1: Implementing K-means

K-means is a relatively simple algorithm that we can write ourselves. For Questions 1 and 2, do not use an existing function that performs K-means directly.

Generate 10 observations with two independent standard normal coordinates. Set a random seed and report it. Randomly assign the observations to $K = 2$ clusters, making sure neither cluster is empty. Let $C(i)$ denote the cluster assigned to observation $x_i$, and let $m_k$ denote the mean of cluster $k$.

a. Perform one iteration: calculate the cluster means, then assign each observation to its closest mean using Euclidean distance. Report the means and updated assignments, and plot the observations colored by their updated assignments.

b. Write a function that repeats these two steps until the assignments stop changing or a specified maximum number of iterations is reached. Your function should work for a data matrix with any number of variables and a chosen number of clusters $K$. Start from a nonempty assignment to each cluster; if a cluster becomes empty during iteration, keep its previous mean. Recalculate the cluster means from the final assignments before returning them. Return the final assignments, cluster means, and within-cluster sum of squares,

$$\sum_{k=1}^{K} \sum_{i:\, C(i)=k} \|x_i - m_k\|^2.$$

c. Apply your function to the same 10 observations from two different random initial assignments. Report the final plots, cluster means, and within-cluster sums of squares. Do the two runs produce the same grouping and objective value? Explain your findings.

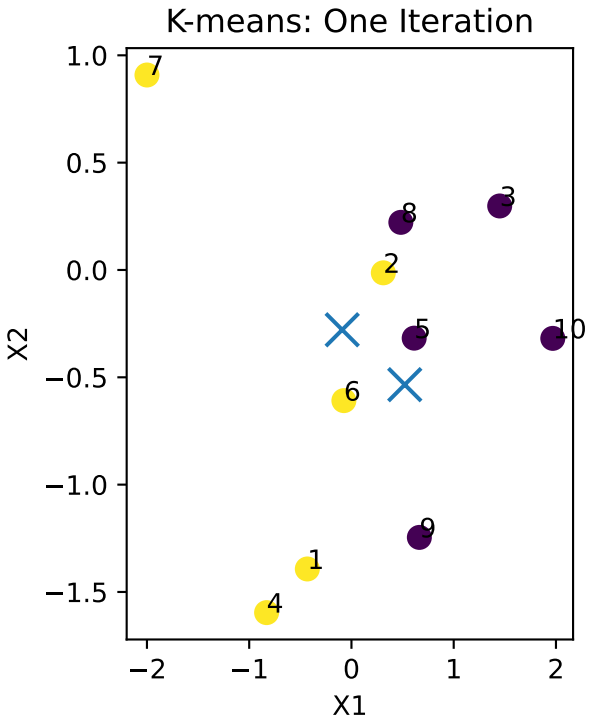
### Answer

### Part a

```
Seed: 2026
Initial assignments:
[1 1 1 2 1 2 2 2 1 2]

Cluster means:
[[ 0.52140537 -0.53427201]
 [-0.09038134 -0.27866384]]

Updated assignments:
[2 2 1 2 1 2 2 1 1 1]
```




**Part b**

```
def my_kmeans(X, K, initial, max_iter=100):

    X = np.asarray(X)
    clusters = np.asarray(initial).copy()

    n, p = X.shape

    # Check initial assignment
    if len(clusters) != n:
        raise ValueError(
            "Initial assignment must have one value per observation."
        )

    if not np.all(np.isin(clusters, np.arange(1, K + 1))):
        raise ValueError(
            "Cluster labels must be between 1 and K."
        )

    for k in range(1, K + 1):
        if np.sum(clusters == k) == 0:
            raise ValueError(
                "Each cluster must be nonempty initially."
            )

    # Initial cluster means
    means = np.vstack([
        X[clusters == k].mean(axis=0)
        for k in range(1, K + 1)
    ])

    for _ in range(max_iter):

        # Distances from each observation to each cluster mean
        distances = np.column_stack([
            np.sum((X - means[k]) ** 2, axis=1)
            for k in range(K)
        ])

        # Assign to closest mean
        new_clusters = np.argmin(distances, axis=1) + 1

        # Stop if assignments do not change
        if np.array_equal(new_clusters, clusters):
            break
```

```
        clusters = new_clusters

        # Update cluster means
        for k in range(1, K + 1):

            # If cluster is nonempty, update its mean
            if np.any(clusters == k):
                means[k - 1] = X[clusters == k].mean(axis=0)

            # If empty, previous mean is retained

    # Recalculate means using final assignments
    for k in range(1, K + 1):
        if np.any(clusters == k):
            means[k - 1] = X[clusters == k].mean(axis=0)

    # Within-cluster sum of squares
    wcss = 0

    for k in range(1, K + 1):
        if np.any(clusters == k):
            differences = X[clusters == k] - means[k - 1]
            wcss += np.sum(differences ** 2)

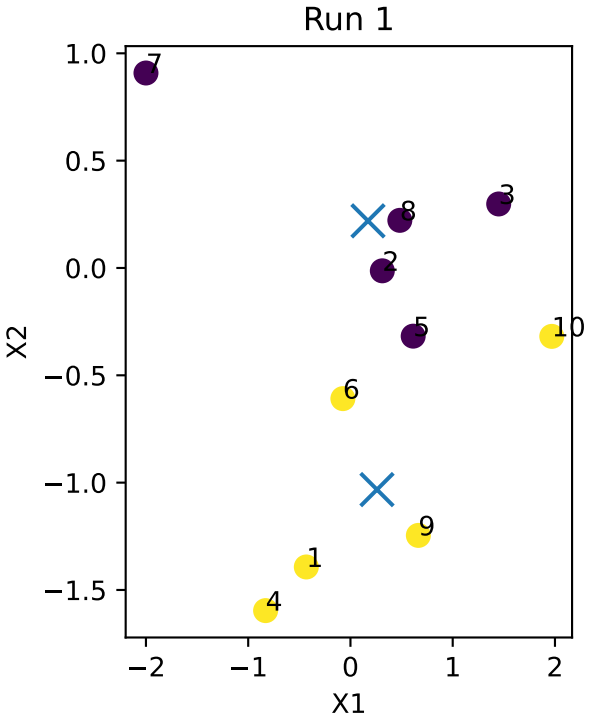
    return {
        "cluster": clusters,
        "centers": means,
        "wcss": wcss
    }
```

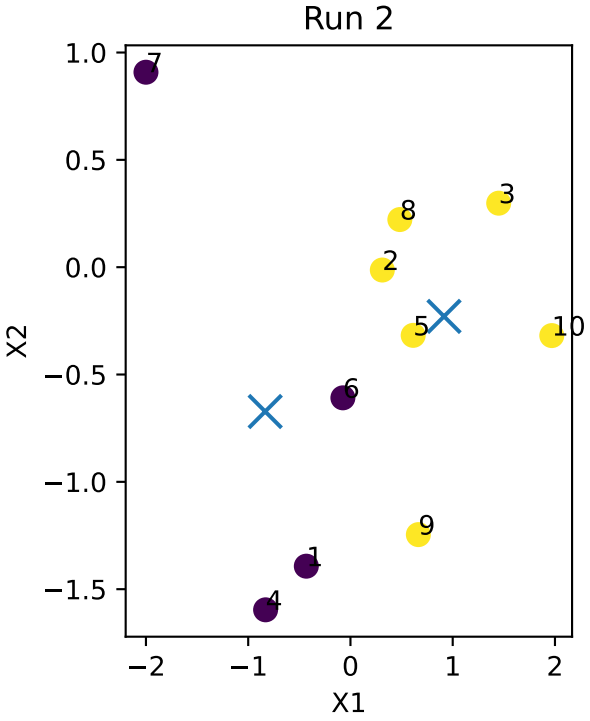
**Part c**

```
Run 1
Initial assignments:
[2 1 2 2 1 1 1 2 2 1]
Final assignments:
[2 1 1 2 1 2 1 1 2 2]
Cluster means:
[[ 0.17156462  0.21948798]
 [ 0.25945941 -1.03242383]]
Within-cluster sum of squares:
13.527554675524954

Run 2
Initial assignments:
[1 2 2 1 2 2 1 1 1 2]
Final assignments:
[1 2 2 1 2 1 1 2 2 2]
Cluster means:
[[-0.83387105 -0.67235568]
 [ 0.91510073 -0.22920942]]
Within-cluster sum of squares:
9.652402157694809

Same grouping: False
Same objective value: False
```




The two runs do not produce the same grouping or the same objective value. Run 1 has a within-cluster sum of squares of **13.53**, while Run 2 has a smaller value of **9.65**.

This happens because K-means can converge to different local minima depending on the initial cluster assignments. Both runs converged in two iterations, but they reached different final solutions. Among all possible two-cluster splits of these 10 observations, **9.65 is the smallest possible WCSS**, so Run 2 found the global optimum, while Run 1 converged to a local minimum.

### Question 2: Image color compression

Pick your favorite color image and use your K-means function from Question 1 to cluster its pixels. For computational efficiency, resize the image to no more than $200 \times 200$ pixels and use its three RGB color channels.

a. Plot the original image and report its dimensions. Transform it into a data matrix with one observation per pixel and three variables: red, green, and blue. Report the dimensions of this matrix.

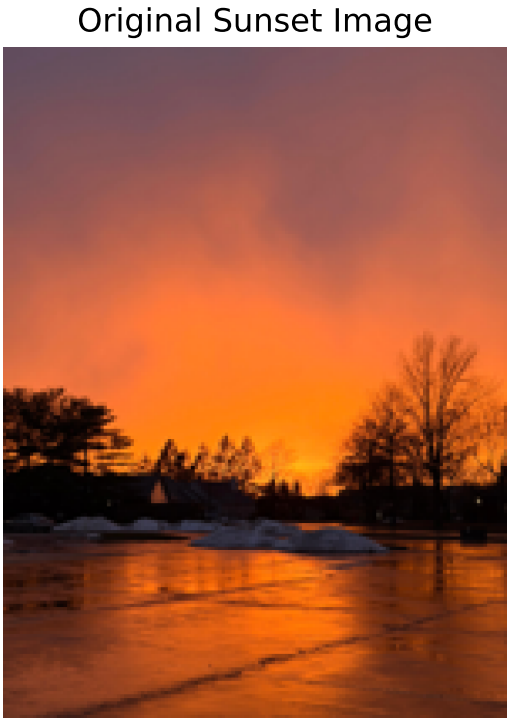
b. For $K = 2, 3, 5, 10$, apply your function to the pixel data. Report the cluster means and within-cluster sum of squares for each $K$. Reconstruct and plot each compressed image by replacing every pixel with its assigned cluster mean. Use the same RGB scale for all fits.

c. Which value of $K$ would you choose for this image? Briefly justify your choice using the reconstructed images and the within-cluster sums of squares.

**Answer**

**Part a**

```
Image dimensions: (200, 150, 3)
Height: 200, Width: 150, Channels: 3
Pixel data matrix dimensions: (30000, 3)
```




I selected a color photograph of a sunset. I resized the image so that neither dimension exceeded 200 pixels while preserving its aspect ratio. The resized image has dimensions $\mathbf{200 \times 150 \times 3}$, corresponding to a height of 200 pixels, a width of 150 pixels, and three RGB color channels.

I scaled the RGB values to the interval $[0, 1]$ and reshaped the image into a data matrix with one observation per pixel and three variables corresponding to red, green, and blue. Since the image contains $200 \times 150 = 30{,}000$ pixels, the resulting pixel data matrix has dimensions

$$30{,}000 \times 3.$$

The original resized sunset image is shown above.

**Part b**

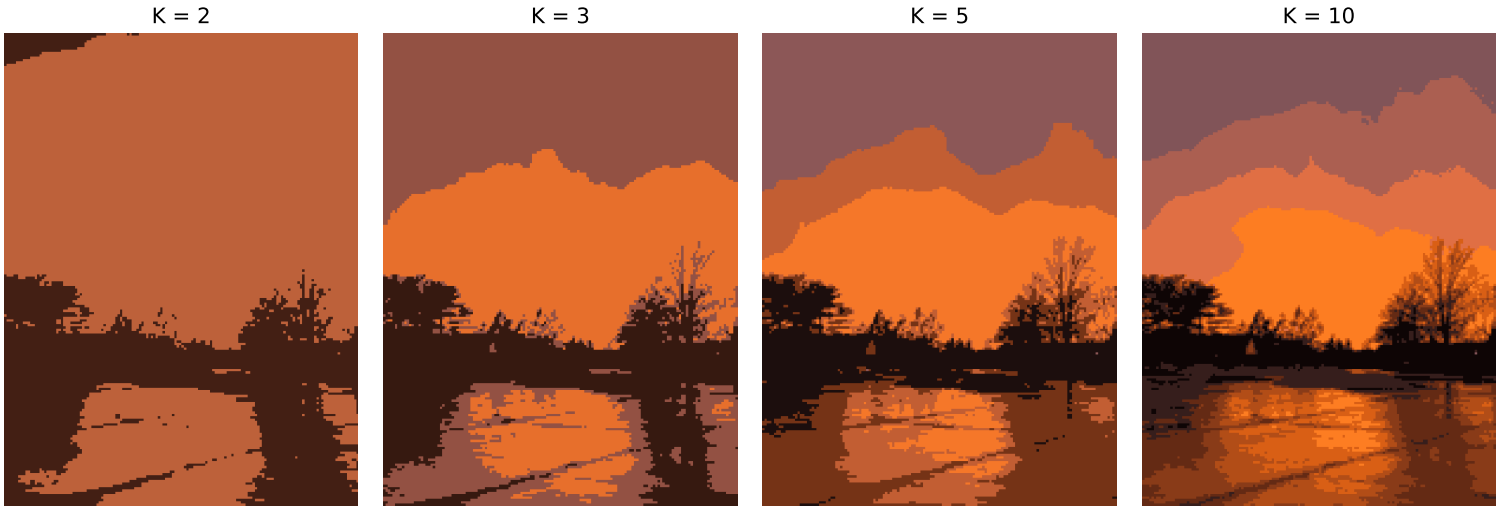
```
------------------------------
K = 2
------------------------------
Cluster means:
[[0.26385662 0.12494755 0.08183366]
 [0.74498729 0.38428923 0.23086759]]
Within-cluster sum of squares:
1415.7356917115767
Cluster sizes:
[8224, 21776]

------------------------------
K = 3
------------------------------
Cluster means:
[[0.5768392  0.32073802 0.26650593]
 [0.21289671 0.09834481 0.06485452]
 [0.9060689  0.4378325  0.17545235]]
Within-cluster sum of squares:
```

```
698.7288917017647
Cluster sizes:
[12809, 6596, 10595]

----------------------------
K = 5
----------------------------
Cluster means:
[[0.96183113 0.46767283 0.16328168]
 [0.46177678 0.20019704 0.08922316]
 [0.11167274 0.05872823 0.05452553]
 [0.76453371 0.36897369 0.20343973]
 [0.54916774 0.34180614 0.34222253]]
Within-cluster sum of squares:
340.20471303036067
Cluster sizes:
[6848, 5115, 4115, 6218, 7704]

----------------------------
K = 10
----------------------------
Cluster means:
[[0.50676408 0.33104085 0.34866997]
 [0.99256721 0.49041287 0.13348044]
 [0.70022591 0.30307956 0.0910821 ]
 [0.87913621 0.43551264 0.26919999]
 [0.39298206 0.16839297 0.08185692]
 [0.67319079 0.37452845 0.32043155]
 [0.53808069 0.23429799 0.09675054]
 [0.05521972 0.0230986  0.0207093 ]
 [0.85216342 0.3740495  0.0854576 ]
 [0.21180887 0.12061322 0.11194719]]
Within-cluster sum of squares:
120.75515382279016
Cluster sizes:
[5137, 4052, 2005, 3661, 2609, 4311, 2118, 2567, 1853, 1687]
```




**Part c**

I would choose $K = 10$ for this image. The reconstructed image with $K = 10$ is visually the closest to the original, preserving more of the smooth color changes in the sky, the dark tree silhouettes, and the reflections on the ground. The within-cluster sum of squares also decreases from 340.21 for $K = 5$ to 120.76 for $K = 10$, showing a substantial improvement in representing the original RGB values.

Although smaller values of $K$ provide greater color compression, $K = 10$ gives the best balance for this image because it still uses only 10 representative colors while retaining much more of the visual detail.

## Question 3: Hierarchical clustering of cars

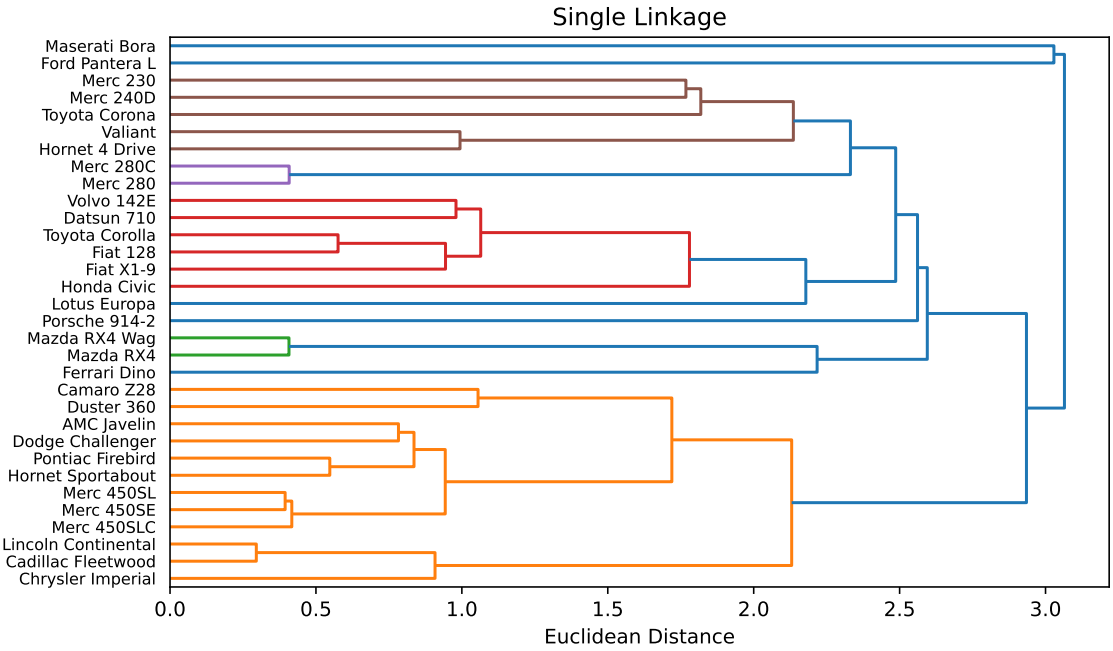
The file `data/mtcars.csv` contains 11 numerical variables for 32 cars, including fuel economy, horsepower, and weight. The `car` column gives the model names; use it to label observations, not as a clustering variable. You may use existing hierarchical clustering functions for this question.

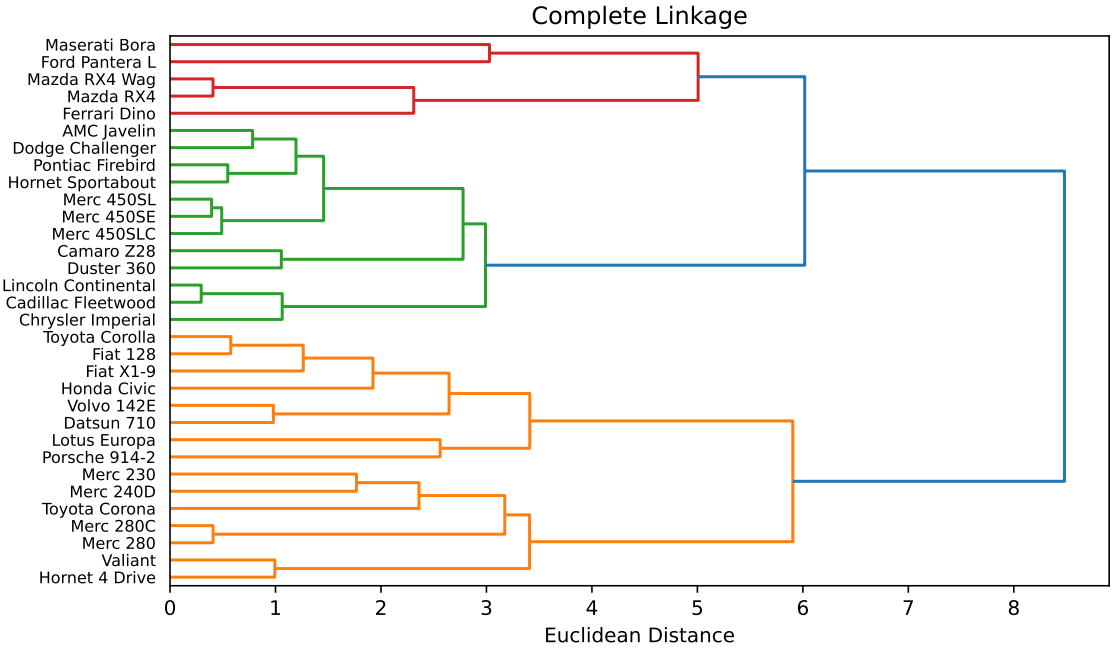
a. Standardize each numerical variable to have mean 0 and standard deviation 1. Using Euclidean distance, perform hierarchical clustering with single, complete, and average linkage. Plot the three dendrograms with the car names and compare the results.

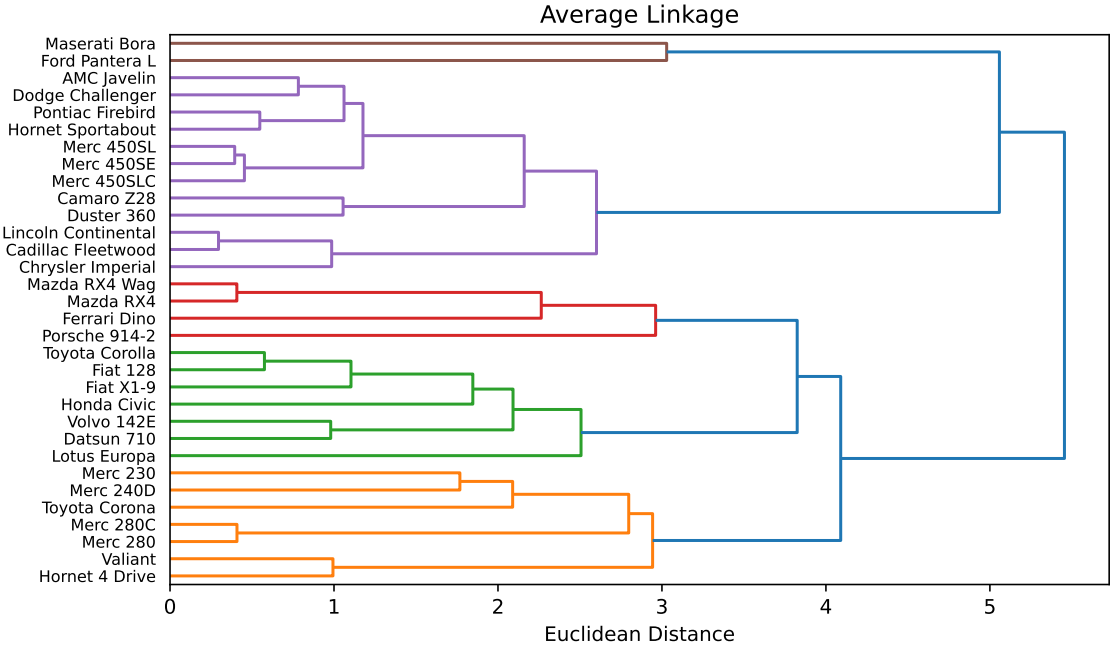
b. Choose one linkage method and a number of clusters that you consider reasonable. Report the cluster memberships and a table of cluster means on the original variable scales. Describe the main characteristics of the groups and explain your choice using the dendrogram and table. Briefly explain why standardizing the variables matters for this analysis.

**Answer**

**Part a**


Single Linkage
Maserati Bora
Ford Pantera L
Merc 230
Merc 240D
Toyota Corona
Valiant
Hornet 4 Drive
Merc 280C
Merc 280
Volvo 142E
Datsun 710
Toyota Corolla
Fiat 128
Fiat X1-9
Honda Civic
Lotus Europa
Porsche 914-2
Mazda RX4 Wag
Mazda RX4
Ferrari Dino
Camaro Z28
Duster 360
AMC Javelin
Dodge Challenger
Pontiac Firebird
Hornet Sportabout
Merc 450SL
Merc 450SE
Merc 450SLC
Lincoln Continental
Cadillac Fleetwood
Chrysler Imperial
0.0
0.5
1.0
1.5
2.0
2.5
3.0
Euclidean Distance


Complete Linkage
Maserati Bora
Ford Pantera L
Mazda RX4 Wag
Mazda RX4
Ferrari Dino
AMC Javelin
Dodge Challenger
Pontiac Firebird
Hornet Sportabout
Merc 450SL
Merc 450SE
Merc 450SLC
Camaro Z28
Duster 360
Lincoln Continental
Cadillac Fleetwood
Chrysler Imperial
Toyota Corolla
Fiat 128
Fiat X1-9
Honda Civic
Volvo 142E
Datsun 710
Lotus Europa
Porsche 914-2
Merc 230
Merc 240D
Toyota Corona
Merc 280C
Merc 280
Valiant
Hornet 4 Drive
0
1
2
3
4
5
6
7
8
Euclidean Distance


Average Linkage
Maserati Bora
Ford Pantera L
AMC Javelin
Dodge Challenger
Pontiac Firebird
Hornet Sportabout
Merc 450SL
Merc 450SE
Merc 450SLC
Camaro Z28
Duster 360
Lincoln Continental
Cadillac Fleetwood
Chrysler Imperial
Mazda RX4 Wag
Mazda RX4
Ferrari Dino
Porsche 914-2
Toyota Corolla
Fiat 128
Fiat X1-9
Honda Civic
Volvo 142E
Datsun 710
Lotus Europa
Merc 230
Merc 240D
Toyota Corona
Merc 280C
Merc 280
Valiant
Hornet 4 Drive
0
1
2
3
4
5
Euclidean Distance

The 11 numerical variables were standardized to have mean 0 and standard deviation 1 before computing Euclidean distances. I then performed hierarchical clustering using single, complete, and average linkage.

The three dendrograms show similar broad groupings, but their merging patterns differ. Single linkage exhibits more of a chaining effect, in which observations or small groups are joined sequentially at relatively small distances. Complete linkage produces more clearly separated and compact groups because it bases each merge on the largest distance between observations in the two clusters. Average linkage gives an intermediate result, with less chaining than single linkage but somewhat less separation than complete linkage.

### Part b

```
Cluster memberships:

Cluster 1:
Hornet 4 Drive, Valiant, Merc 240D, Merc 230, Merc 280, Merc 280C, Toyota Corona

Cluster 2:
Datsun 710, Fiat 128, Honda Civic, Toyota Corolla, Fiat X1-9, Porsche 914-2, Lotus Europa, Volvo
142E

Cluster 3:
Hornet Sportabout, Duster 360, Merc 450SE, Merc 450SL, Merc 450SLC, Cadillac Fleetwood, Lincoln
Continental, Chrysler Imperial, Dodge Challenger, AMC Javelin, Camaro Z28, Pontiac Firebird

Cluster 4:
Mazda RX4, Mazda RX4 Wag, Ford Pantera L, Ferrari Dino, Maserati Bora

Cluster means on original scales:
           mpg   cyl    disp      hp  drat    wt   qsec    vs   am  gear  carb
Cluster
1        20.74  5.14  175.11  102.14  3.57  3.19  19.97  1.00  0.0  3.57  2.14
2        28.08  4.00   93.61   81.88  4.18  2.04  18.45  0.88  1.0  4.25  1.50
3        15.05  8.00  357.62  194.17  3.12  4.10  17.14  0.00  0.0  3.00  3.08
4        18.50  6.80  223.40  198.80  3.84  3.00  15.62  0.00  1.0  4.60  5.20
```

I chose **complete linkage with four clusters**. The complete-linkage dendrogram shows relatively clear separation between four major groups before substantially larger-distance merges occur.

Cluster 1 contains mostly moderate-sized cars with moderate fuel economy and relatively low horsepower. Its mean fuel economy is 20.74 mpg and mean horsepower is about 102.

Cluster 2 contains the smallest and most fuel-efficient cars. These cars average 28.08 mpg, have four cylinders on average, low displacement and horsepower, and relatively low weight. All of them are manual-transmission cars.

Cluster 3 consists of large, heavy, high-displacement cars, all 12 with eight cylinders. This group has the lowest average fuel economy, 15.05 mpg, and the highest average weight, about 4.10, along with high horsepower.

Cluster 4 contains relatively high-performance manual-transmission cars. These cars have high average horsepower, 198.8, relatively low quarter-mile times, and the largest average number of gears and carburetors.

Standardizing the variables is important because the variables are measured on very different numerical scales. For example, displacement and horsepower have much larger numerical ranges than variables such as transmission type or number of cylinders. Without standardization, variables with larger scales would dominate the Euclidean distances and therefore have too much influence on the clustering.